

# Validation State Continuity & Lifecycle Record Module (VSCLRM)

**Architecture Ecosystem:** Structured Reality Standards™

**Architecture Family:** VALIDOS® — Validation Governance Architecture

**Parent Standard:** Validation State Governance Standard (VSGS)

**Operational Layer:** Validation State Governance Layer

**Category:** AI & Interpretation

**Subcategory:** Validation Governance

**Type:** Parent Standard Module

**Governed Space:** Validation State Continuity & Lifecycle Record

**Version:** 1.0

**Status:** Canonical · Open Module

**Effective Date:** 12 August 2026

**Compatibility:** OOF® Methodology OS™ · GOA™ · OBIDENITY® · INTEGROS®  
· ORA™ · AGA™ · AIG® · CLIA® · MGIA™ · ASGA™ · RIS™

**AI-Readable:** Yes

**Authority:** OOF®

**Protection:** MIP® — Methodological Intellectual Property

**Canonical Language:** English (UCL™)

---

## Governed Space

**Validation State Continuity & Lifecycle Record**

---

## Canonical Definition

### Validation State Continuity & Lifecycle Record Module (VSCLRM)

defines the governance framework for preserving, linking, versioning, reconstructing, and maintaining the complete lifecycle history of Validation States, including their assignments, applicability conditions, transitions, restrictions, suspensions, resumptions, expirations, invalidations, conflicts, supersessions, withdrawals, revalidation requirements, and relationships to the Validation Determinations and Validation Objects from which those states derive.

It establishes the historical continuity layer through which VALIDOS® preserves both:

**what Validation State the object holds now**

and:

**how, why, and from which governed validation history that state emerged.**

---

## Operational Role

VSCLRM serves as the fifth and final internal governance space of the Validation State Governance Standard.

The preceding modules establish:

**VSAEM — Which Validation State may legitimately be assigned?**

**VSCAM — Where, when, and under which conditions does that state apply?**

**VMCSMM — What has changed, and does that change materially affect the state?**

**VSTRM — How should the state transition, and when is revalidation required?**

VSCLRM asks:

**Can the complete Validation State lifecycle be reconstructed across time without losing the determinations, conditions, changes, transitions, and historical states that produced the object's current validation condition?**

The governed progression is:

**State Assignment → Active State → Applicability History → Change Signals → State Transition → Revalidation Trigger → New State → Lifecycle Continuity → Current & Historical Validation State Record**

VSCLRM closes Validation State Governance.

---

## Module Operational Space

VSCLRM governs:

- Validation State continuity,
- Validation State history,
- state lifecycle records,
- state assignment history,
- state applicability history,
- state-condition history,
- state restriction history,
- state-limitation history,
- state transition history,
- state suspension history,
- state resumption history,
- state expiration history,
- state invalidation history,
- state conflict history,
- revalidation-required history,
- state supersession,

- state withdrawal,
- state versioning,
- state lineage,
- state chronology,
- determination-to-state linkage,
- state-to-object linkage,
- object-version continuity,
- revalidation-cycle linkage,
- historical-state preservation,
- current-state identification,
- lifecycle reconstruction,
- lifecycle-record completeness,
- lifecycle-record correction,
- lifecycle-record integrity awareness,
- and state-lifecycle traceability.

---

## Module Function

VSCLRM applies throughout the complete Validation State lifecycle.

Its function is to prevent:

- current Validation State being detached from the validation history that created it,
- old states being overwritten,
- state transitions occurring without historical linkage,
- revalidation cycles appearing as independent events,
- superseded determinations disappearing,
- suspended and resumed states losing temporal continuity,
- state conditions changing without historical record,
- invalidated validation being erased,
- current status being confused with historical status,
- different object versions being merged into one validation history,
- and validation claims becoming impossible to reconstruct after repeated lifecycle changes.

---

## Validation State Continuity

**Validation State Continuity** is the preserved relationship between successive Validation States of the same Validation Object or governed object lineage.

Continuity answers:

**What state existed before?**

**What caused it to change?**

**What state replaced it?**

**Which Validation Determination supported each state?**

**Which object version did each state apply to?**

**Which conditions and restrictions were active?**

**What is the current state now?**

---

## Historical Validation State

A **Historical Validation State** is a state that legitimately applied to the Validation Object during a defined prior lifecycle period.

Historical state is not current state.

But it remains part of the methodological record.

For example:

**1 January–30 June: Validated**

**1 July–10 July: Validation Suspended**

**11 July–Present: Conditionally Validated**

All three states may be historically legitimate.

---

## Current Validation State

The **Current Validation State** is the formal state currently applicable to the Validation Object within the governed time, scope, version, and conditions.

VSCLRM ensures that the current state can be distinguished from every previous state.

---

## Current State Is Not the Latest Label

A state should not be considered current merely because it is the most recently entered record.

Current-state identification should consider:

- effective point,
  - expiration,
  - supersession,
  - suspension,
  - withdrawal,
  - object version,
  - scope,
  - and applicable transition rules.
-

## State Chronology

VSCLRM preserves the temporal ordering of state events.

A lifecycle may appear as:

### Validation Required



### Validation Pending



### Validated



### Conditionally Validated



### Validation Suspended

### ↓ Revalidation Required



### Validation Pending



### Validated

Chronology creates the time dimension of validation governance.

---

## State Lineage

State Lineage connects one state to the governance basis from which it emerged.

A state lineage may include:

### Validation Object



### Validation Determination



### State Eligibility



## State Assignment



## State Conditions



## Material Change



## State Transition



## Successor State

This creates a reconstructable validation-state chain.

---

## Determination-to-State Linkage

Each materially assigned Validation State should remain connected to the Validation Determination supporting it.

Where the state changes without a new determination, the transition basis should remain visible.

For example:

### Validated → Validation Expired

may result from expiration logic rather than a new determination.

The historical Positive Validation Determination remains linked to the earlier Validated state.

---

## State-to-Object Linkage

Every state must remain attached to the correct Validation Object and version.

Where an object changes materially, the state history should preserve:

### Object Version 1 → Validation State A

### Object Version 2 → Validation State B

rather than representing both as one undifferentiated state record.

---

## Object Lineage and Validation State

A Validation Object may evolve through versions.

VSCLRM may preserve:

**Object 1.0 → Object 1.1 → Object 2.0**

together with the Validation States corresponding to each version.

This permits later assessment of how validation evolved with the object.

---

## State Versioning

Material changes to Validation State records should remain versioned.

For example:

**State Record 1.0**

**State Record 1.1**

**State Record 2.0**

Versioning should distinguish:

- correction,
  - transition,
  - scope change,
  - condition change,
  - and new state assignment.
- 

## State Record Update Is Not State Transition

VALIDOS® distinguishes:

**State Record Update**

from:

**Validation State Transition**

Adding explanatory metadata does not necessarily change the state.

Changing:

**Validated**

to: **Conditionally Validated**

does.

---

This distinction prevents administrative updates from being confused with methodological lifecycle changes.

---

## State Assignment History

VSCLRM preserves each formal state assignment together with:

- assignment basis,
  - object version,
  - determination,
  - scope,
  - depth,
  - conditions,
  - restrictions,
  - effective point,
  - and authority where applicable.
- 

## Applicability History

The applicability of a state may change without immediate state transition.

For example:

A state may initially apply to:

### Environment A + B

and later be restricted to:

### Environment A

VSCLRM preserves the historical Applicability Profile rather than storing only the latest boundary.

---

## Condition History

State conditions may be:

- introduced,
- modified,
- satisfied,
- violated,
- restored,
- or removed through governed reassessment.

The history of these conditions may explain later state transitions.

---



## Restriction History

Restrictions should remain historically visible.

For example:

### Not Validated for Autonomous Operation

may later be removed after successful revalidation.

The fact that the earlier state carried the restriction should remain part of the historical record.

---

## Limitation History

Known limitations may evolve.

A limitation may be:

- reduced by new evidence,
- increased after an incident,
- resolved,
- superseded,
- or transformed into a formal restriction.

VSCLRM preserves this evolution.

---

## Transition History

Every material Validation State transition should preserve:

### Previous State

### Transition Trigger

### Transition Basis

### Affected Scope

### New State

### Effective Point

### Authority

and:

### Revalidation Requirement

where applicable.

---

## Suspension History

Where validation is suspended, the lifecycle record should preserve:

- suspension trigger,
  - affected scope,
  - suspension date,
  - active investigation or condition,
  - prior state,
  - and later resolution.
- 

## Resumption History

Resumption should preserve:

- the basis for restoring validation,
  - whether the original state was restored,
  - whether the state became conditional or partial,
  - and which conditions changed.
- 

## Expiration History

Expiration records should distinguish between:

### State Expired

and:

### Object Failed

The historical record should show that validation ceased to be current because the validity model ended.

---

## Invalidation History

Invalidation is a major lifecycle event.

VSCLRM preserves:

- invalidated state,
  - invalidation basis,
  - affected scope,
  - date,
  - relevant determination,
  - evidence,
  - source or method failure where applicable,
  - and subsequent remediation or revalidation.
-

Invalidation should never disappear merely because the object later becomes validated again.

---

## Revalidation History

Revalidation cycles should remain connected to the states that triggered them.

The conceptual lifecycle may be:

### Validated

↓

### Material Change

↓

### Revalidation Required

↓

### Validation Pending

↓

### New Validation Determination

↓

### Validated Version 2

VSCLRM makes this a continuous lifecycle rather than separate records.

---

## Revalidation Cycle Identifier

A revalidation cycle may receive a distinct identifier linking:

- trigger,
  - previous state,
  - affected scope,
  - new validation process,
  - new determination,
  - and successor state.
- 

## State Supersession

A state is superseded where a newer governed state replaces it for the same applicable domain.

---

For example:

## Conditionally Validated — Version 3.0

may be superseded by:

## Validated — Version 3.1

Supersession does not delete the earlier state.

---

## Partial Supersession

A new state may supersede only part of an earlier state.

For example:

## System Validated for A + B

may later become:

### A — Validated

### B — Validation Suspended

The original state is only partially superseded.

---

## State Withdrawal

A Validation State may be withdrawn where it should never have been assigned or where its assignment basis is discovered to be invalid.

Withdrawal differs from invalidation.

**Invalidation** means a previously legitimate state later lost its basis.

**Withdrawal** may mean the state assignment itself was erroneous or illegitimate.

---

## Withdrawal History

Withdrawal should preserve:

- withdrawn state,
  - original assignment,
  - reason,
  - affected period,
  - correction,
  - and replacement state if applicable.
-

## State Conflict History

Where an object enters **Validation Conflicted**, VSCLRM preserves:

- competing state signals,
  - affected scope,
  - relevant determinations,
  - conflict period,
  - resolution,
  - and successor state.
- 

## Concurrent Validation States

Complex objects may possess multiple simultaneous state dimensions.

For example:

**Performance — Validated**

**Safety — Conditionally Validated**

**Autonomy — Revalidation Required**

VSCLRM preserves these concurrent state histories separately while maintaining their relationship to the same Validation Object.

---

## State Dimension Identity

Each concurrent state dimension should be identifiable.

Possible dimensions include:

- capability,
  - subsystem,
  - function,
  - population,
  - environment,
  - jurisdiction,
  - operational mode,
  - risk domain,
  - or another governed category.
- 

## Hierarchical State Continuity

Validation State may exist across:

**Component → Subsystem → System**

VSCLRM preserves hierarchical relationships across

---

lifecycle changes.

A component-state change may propagate upward where the higher-level state depends upon it.

---

## State Propagation History

Where one state change influences another state, the relationship should remain traceable.

For example: **Critical Component Invalidated**

↓

### System Validation Suspended

This helps later reviewers understand why the system-level state changed.

---

## Lifecycle Event

A **Validation Lifecycle Event** is a material event affecting Validation State continuity.

Events may include:

- state assignment,
- state activation,
- applicability change,
- condition violation,
- material change,
- suspension,
- resumption,
- expiration,
- invalidation,
- revalidation trigger,
- revalidation start,
- new determination,
- state transition,
- supersession,
- or withdrawal.

---

## Lifecycle Event Identity

A lifecycle event may record:

- Event Identifier,
- Validation Object,
- state before event,
- event type,
- trigger,
- time,

- affected scope,
- governing evidence,
- authority,
- resulting state,
- and related events.

---

## State Timeline

VSCLRM may create a **Validation State Timeline** representing the complete chronological lifecycle of the Validation Object.

The timeline may display:

### Object Version

### State

### Scope

### Conditions Trigger

### Determination

### Effective Period

and:

### Successor State

---

## Validation State History Map

A more complex object may require a **Validation State History Map** rather than a linear timeline.

This may represent branching state histories across:

- versions,
- components,
- populations,
- environments,
- or operational modes.

---

## Lifecycle Reconstruction

A qualified reviewer should be able to reconstruct:

### What was the object's state at Time T?

**Which object version did that state apply to?**

**Which determination supported it?**

**Which conditions applied?**

**Was the state suspended?**

**Was it expired?**

**What caused the next transition?**

**Was revalidation performed?**

**What state is current now?**

---

## **Point-in-Time State Query**

VSCLRM conceptually enables a point-in-time validation query:

**What Validation State did Object X hold on Date Y under Scope Z?**

This is valuable for:

- audit,
  - incident investigation,
  - regulatory review,
  - liability analysis,
  - operational governance,
  - and historical decision reconstruction.
- 

## **Current-State Query**

The lifecycle record should also support:

**What Validation State does Object X hold now?**

together with:

- scope,
  - conditions,
  - restrictions,
  - expiration,
  - version,
  - and applicable determination.
- 

## **Historical Validation Claim Verification**

---



A historical claim such as:

**“The system was validated when this decision was made.”**

can be tested against the State Lifecycle Record.

This turns validation history into auditable methodological evidence.

---

## Lifecycle Record Completeness

The Validation State Lifecycle Record may be:

**Lifecycle Record Complete**

**Conditionally Complete**

**Partially Complete**

**Incomplete**

or:

**Completeness Undetermined**

Completeness concerns whether material state history can be sufficiently reconstructed.

---

## Lifecycle Gap

A **Lifecycle Gap** exists where a material period, transition, trigger, determination relationship, or state condition cannot be sufficiently reconstructed.

Examples include:

- unknown state during one period,
  - missing transition authority,
  - absent revalidation link,
  - unknown object version,
  - missing suspension reason,
  - or unknown applicability conditions.
- 

## Critical Lifecycle Gap

A Lifecycle Gap becomes critical where it prevents meaningful determination of:

- what state applied,
  - whether reliance was potentially legitimate,
-

- whether validation was current,
- or whether a transition occurred.

Critical gaps become important inputs to the final VALIDOS® reliance layer.

---

## Lifecycle Record Correction

Historical records may occasionally require correction.

A correction should preserve:

**Original Record → Identified Error → Corrected Record → Reason → Correction Time**

Historical evidence should not be silently rewritten.

---

## Lifecycle Record Integrity Awareness

VSCLRM depends upon trustworthy lifecycle records.

It does not duplicate INTEGROS®.

Where record integrity is materially uncertain, that uncertainty should remain attached to the lifecycle record.

---

## Lifecycle Record Retention

Validation State records may require long retention in high-consequence environments.

Retention requirements may depend upon:

- lifecycle duration,
- regulatory requirements,
- incident exposure,
- audit requirements,
- accountability,
- or historical reliance needs.

VSGS does not prescribe one universal retention period.

---

## Lifecycle Record Portability

Where a Validation Object moves between:

- organizations,
  - systems,
  - operators,
-

- jurisdictions,
- or infrastructure,

its Validation State history may need to remain portable.

Portability does not automatically transfer operational reliance rights.

It preserves validation history.

---

## Lifecycle Record Interoperability

Machine-readable lifecycle records may support interoperability across systems.

An interoperable record might allow an external governance system to understand:

- current state,
  - state history,
  - object version,
  - determination references,
  - restrictions,
  - expiration,
  - and revalidation history.
- 

## State Identity Across Systems

The same Validation State should not acquire different meanings merely because it is represented in different platforms.

VSCLRM therefore supports canonical state semantics across interoperable systems.

---

## Machine-Readable Validation Lifecycle

A machine-readable lifecycle may represent:

**Object ID**

**Version**

**State**

**Effective From**

**Effective Until**

**Scope**

---

**Conditions**

**Restrictions**

**Determination Reference**

**Trigger**

**Previous State**

**Next State**

**Revalidation Cycle**

This creates the basis for automated validation-history reasoning.

---

## AI-Assisted Lifecycle Reconstruction

AI may assist with:

- timeline reconstruction,
- state-record comparison,
- transition identification,
- missing-link detection,
- determination-to-state mapping,
- anomaly detection,
- and historical summary.

AI should not fabricate missing lifecycle events.

Unresolved gaps must remain explicit.

---

## Automated Current-State Resolution

Where the lifecycle rules are explicit, a system may automatically resolve the current state from:

- latest applicable transition,
- state effective periods,
- expiration,
- supersession,
- suspension,
- object version,
- and active conditions.

Automated state resolution must remain traceable.

---

## Lifecycle Continuity and Accountability

---

State continuity helps establish which validation condition existed when:

- deployment occurred,
- an operational decision was made,
- an incident occurred,
- reliance was authorized,
- or a system changed.

This creates strong integration potential with AGA™ and other accountability architectures.

---

## Lifecycle Continuity and Reliance

VSCLRM does not authorize reliance.

However, the final VALIDOS® Parent Standard will need to know:

- what state was current,
- whether the state was applicable,
- whether it had expired,
- whether it was suspended,
- whether restrictions applied,
- and whether revalidation was pending

before legitimate reliance can be assessed.

VSCLRM provides that lifecycle evidence.

---

## Lifecycle Continuity and Revalidation

Revalidation should never appear disconnected from prior validation.

The architecture preserves:

### Previous Validation Basis

↓

### State Change Trigger

↓

### Revalidation Requirement ↓

### New Validation Cycle

↓

### New Determination

---



## New Validation State

This makes revalidation a continuation of validation governance rather than a complete methodological reset.

---

## Minimum Implementation Framework

### 1. Establish Validation State Lifecycle Identifier

Create a persistent lifecycle reference for the Validation Object.

### 2. Link State to Validation Object

Preserve object identity, version, configuration, and relevant lineage.

### 3. Record State Assignment

Maintain the state, basis, scope, conditions, restrictions, depth, and effective point.

### 4. Preserve Applicability History

Track changes in scope, conditions, limitations, exclusions, and validity.

### 5. Record Lifecycle Events

Capture material:

- change signals,
- transitions,
- suspensions,
- resumptions,
- expirations,
- invalidations,
- revalidation triggers,
- supersessions,
- and withdrawals.

### 6. Link Revalidation Cycles

Connect each revalidation cycle to the state event that triggered it and to the determination and successor state it produced.

### 7. Preserve Historical States

Never overwrite earlier legitimate Validation States.

### 8. Resolve Current Validation State

Identify which state is currently applicable across the relevant scope and version.

**9. Assess Lifecycle Record Completeness** Identify missing periods, transitions, object versions, or state relationships.

## 10. Preserve Corrections and Versions

Maintain historical versions and correction lineage.

## 11. Support Point-in-Time Reconstruction

Enable reconstruction of the Validation State applicable at any material lifecycle point where records permit.

## 12. Preserve Lifecycle Traceability

Maintain:

- Lifecycle Identifier,
- Validation Object,
- object versions,
- Validation Determinations,
- state assignments,
- state conditions,
- applicability profiles,
- material changes,
- transitions,
- revalidation triggers,
- revalidation cycles,
- suspensions,
- resumptions,
- expirations,
- invalidations,
- conflicts,
- supersessions,
- withdrawals,
- current state,
- historical states,
- lifecycle gaps,
- corrections,
- versions,
- and sufficient lifecycle continuity.

---

## Governance Outputs

VSCLRM may produce:

- Validation State Lifecycle Identifier,
- Validation State Lifecycle Record,
- Validation State History,
- Validation State Timeline,
- Validation State History Map,
- Validation State Lineage Map,

- Validation Determination-to-State Linkage Record,
- Validation Object-to-State Linkage Record,
- Object Version State Map,
- State Assignment History,
- State Applicability History,
- State Condition History,
- State Restriction History,
- State Limitation History,
- State Transition History,
- Validation Suspension History,
- Validation Resumption History,
- Validation Expiration History,
- Validation Invalidation History,
- Validation Conflict History,
- Revalidation History,
- Revalidation Cycle Record,
- State Supersession Record,
- Partial Supersession Record,
- State Withdrawal Record,
- Concurrent Validation State Map,
- Hierarchical State Continuity Map,
- State Propagation Record,
- Validation Lifecycle Event Register,
- Point-in-Time Validation State Record,
- Current Validation State Record,
- Lifecycle Record Completeness Assessment,
- Validation Lifecycle Gap Record,
- Critical Lifecycle Gap Record,
- Lifecycle Record Correction Record,
- Validation State Record Version,
- and Validation State Continuity Reference.

---

## Use Case 1 — Continuously Updated AI System

### Scenario

An AI platform is repeatedly updated over two years.

Its history includes:

- Validated Version 1,
- several minor updates,
- a material capability expansion,
- Revalidation Required,
- Validation Pending,
- Conditionally Validated Version 2,
- later restriction removal,
- and Validated Version 2.3.

### Application

VSCLRM links each object version, state, trigger, determination, applicability profile, and revalidation cycle.



## Result

A reviewer can determine exactly:

- when the system was Validated,
- when it required revalidation,
- when conditional restrictions applied,
- and which Validation State applied to each version.

The latest state does not erase the prior lifecycle.

---

## Use Case 2 — Healthcare AI Incident Review

### Scenario

A healthcare AI system is involved in an incident.

The organization claims the system was Validated at the time.

**Application** VSCLRM reconstructs the point-in-time Validation State.

The record shows:

- a historical Positive Validation Determination,
- subsequent Material Change,
- a Revalidation Required transition,
- and no completed revalidation before the incident.

## Result

The architecture can distinguish:

### Historically validated

from:

### Current state at incident: Revalidation Required

without rewriting the earlier validation history.

---

## Use Case 3 — Industrial System Revalidation

### Scenario

An industrial system undergoes three successive revalidation cycles following hardware, software, and environmental changes.

### Application

---

VSCLRM connects:

**State before change → Change Trigger → Revalidation Cycle → New Determination → Successor State**

for each cycle.

## Result

The validation history becomes one continuous methodological lineage rather than four unrelated validation reports.

---

## Architectural Position

Validation State Continuity & Lifecycle Record is the fifth and final internal governance space of the **Validation State Governance Standard (VSGS)**.

The complete VSGS progression is:

**Validation State Assignment & Eligibility → Validation State Conditions & Applicability → Validation Material Change & State Monitoring → Validation State Transition & Revalidation Triggering → Validation State Continuity & Lifecycle Record**

Together, these five governed spaces transform a Validation Determination into a current and continuously governable Validation State.

The lifecycle becomes:

### Validation Determination

↓

### State Eligibility ↓

### State Assignment

↓

### State Applicability

↓

### State Monitoring

↓

### Material Change

↓

### State Transition

---

↓

## **Revalidation Trigger**

↓

## **New Validation Cycle**

↓

## **Successor Validation State**

↓

## **Lifecycle Continuity**

The ninth Parent Standard is therefore complete.

The final VALIDOS® Parent Standard can now address the last architectural question:

**Who or what may legitimately rely upon a current Validation State, for which purpose, under which conditions, for how long, and how must reliance respond when validation changes or revalidation becomes necessary?**

---

## **Foundational Principle**

**A current Validation State is trustworthy only when its entire lifecycle—from the determination that created it to every change that followed—remains reconstructable.**

---

## **Canonical Closing Statement**

**Validation State Continuity & Lifecycle Record Module completes the lifecycle-state governance layer of VALIDOS® by preserving the complete methodological history through which Validation Objects acquire, maintain, restrict, suspend, expire, invalidate, supersede, withdraw, and regain Validation States across change and revalidation cycles. By governing state lineage, chronology, determination linkage, object-version continuity, applicability history, transition history, revalidation history, concurrent and hierarchical states, lifecycle events, point-in-time reconstruction, current-state resolution, lifecycle gaps, corrections, versioning, and historical preservation, VSCLRM ensures that validation status can never be reduced to an isolated current label and that every present Validation State remains traceable to the complete governed validation lifecycle that legitimately produced it.**

---

# Parent Resources

→ [VALIDOS® Validation Governance Architecture — Validation Governance Layer](#)

→ [VALIDOS® Architecture Map](#)

→ [VALIDOS® Complete Standards & Modules Index](#)

---

## Internal Module Flow

→ [Previous module: Validation State Transition & Revalidation Trigger Module \(VSTRM\)](#)

---

## Related Documents

→ [Validation State Governance Standard](#)

→ [About Validation State Governance Standard](#)

# OOF® MIP® Protection Notice

**OOF® · Protected under MIP® — Methodological Intellectual Property**

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

**Active Protection & Compatibility Detection applies.**

## Canonical Source

<https://originopenfoundation.org/>